Large-Scale Optimization for Transport & Mobility
Assignment 2

Yasmine Beck

Deadline: Monday, October 12, 2026, 17:00

This assignment is to be completed in teams of **two students**. Collaboration between teams is **not** allowed, but helping with minor coding questions is permitted. Copying code or text from other teams is considered plagiarism. You may use external resources, including ChatGPT, Stack Overflow, and Python libraries, but any external resources used must be clearly acknowledged.

Solving a Two-Echelon Capacitated Vehicle Routing Problem using Adaptive Large Neighborhood Search

In this assignment, you will solve the Two-Echelon Capacitated Vehicle Routing Problem (2E-CVRP) by developing an Adaptive Large Neighborhood Search (ALNS) heuristic. The 2E-CVRP considers the transportation of goods from a depot to customers via satellite locations. The objective is to minimize the total traveled distance, the fixed costs of the vehicles used, and the handling costs, while respecting vehicle capacity constraints. Formally, we are given:

- a set of customers, each characterized by a delivery location and a demand;
- a set of depots from which first-echelon vehicles start and end their routes;
- a set of satellite locations from which second-echelon vehicles start and end their routes;
- the cost and capacity of first- and second-echelon vehicles;
- the handling cost at each satellite; and
- the distances between all locations.

An illustration of a 2E-CVRP solution is given in Figure 1. For more information about the problem, please refer to Sluijk et al. (2023).

For this assignment, the number of available vehicles is assumed to be nonrestrictive. The vehicle capacities and costs are the same for all instances and are specified as follows:

- first-echelon vehicles: capacity 200, cost 50;
- second-echelon vehicles: capacity 50, cost 25.

The handling cost per unit demand at each satellite is 5.

Your Task

Your task is to improve the provided Adaptive Large Neighborhood Search algorithm for the Two-Echelon Capacitated Vehicle Routing Problem. The implementation is provided in the *zip-file* `assignment-2.zip`, available on Canvas. The provided template contains a basic ALNS implementation with one destroy operator and one repair operator. Your goal is to extend the template and develop a more effective ALNS algorithm. Your implementation must include the following components:

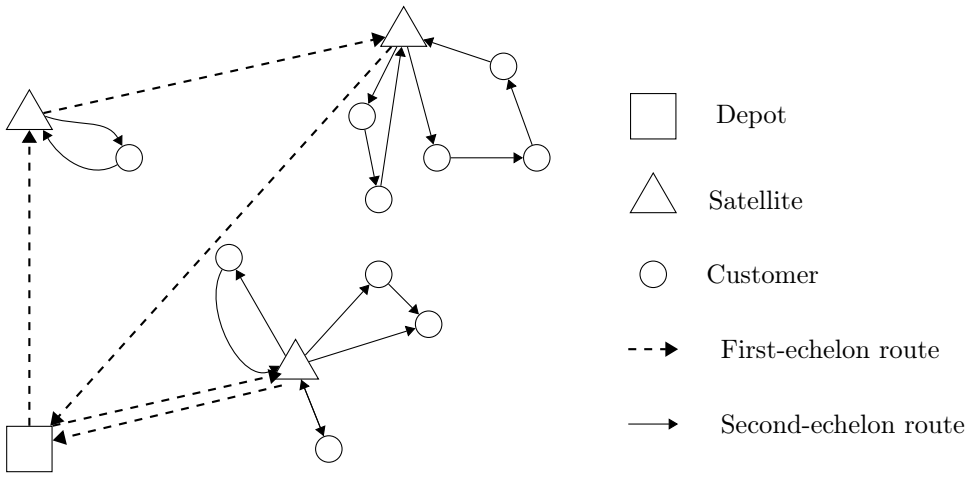


Figure 1: An example of a 2E-CVRP solution (adapted from Fig. 1 in Sluijk et al. (2023)).

1. Implement at least two additional destroy operators and at least two additional repair operators, either based on the literature (see, e.g., Ropke and Pisinger (2006)) or designed by yourselves. As part of this task, use an AI assistant to obtain suggestions for suitable destroy and repair operators for the 2E-CVRP. Critically evaluate these suggestions and decide which operators to implement. Note that you are not required to follow the suggestions provided by the AI.
2. Make the ALNS adaptive by implementing and updating weights according to which the destroy and repair operators are selected.
3. Embed the search in a simulated annealing metaheuristic by modifying the subroutine for the acceptance criterion.

In addition to these **three required components**, you must complete **at least one** of the following additional tasks. Which option you choose is completely up to you.

1. Implement an alternative constructive heuristic to initialize the ALNS.
2. Identify at least two important parameters of your ALNS and tune their values by systematically investigating their effect on the algorithm's performance.
3. Extend the 2E-CVRP by introducing customer time windows and adapt the relevant ALNS components as necessary.

For all adaptations, use the provided template as a starting point and extend or modify it where necessary. You are not expected to rewrite the ALNS framework from scratch.

Template

The provided template contains a basic implementation of an ALNS algorithm with one destroy operator and one repair operator. The template is provided in the *zip-file* `assignment-2.zip`, available on Canvas. For clarity, the code has been distributed over the following files:

- **problem.py**: The Python file that defines the class representing a 2E-CVRP instance.
- **route.py**: The Python file that contains the class that represents a route taken by either a first- or a second-echelon vehicle. The class contains methods to compute the distance and cost of the route.
- **alns.py**: The Python file that contains the actual ALNS class.
- **solution.py**: The Python file that represents a solution to the 2E-CVRP. This class also contains the destroy and repair operators of the ALNS algorithm.
- **main.py**: The Python file that reads the data from a given 2E-CVRP instance and calls the ALNS algorithm. If you run this file, you see the output of the algorithm.

Before starting the implementation, take some time to familiarize yourself with the provided code. You do not need to understand every line of code, but you should understand the overall structure of the implementation and the purpose of the main classes and methods.

Instance Data

The folder `instances` in the *zip-file* `assignment-2.zip`, available on Canvas, contains the 24 instances that you should use for your computational experiments. The instances differ in the number of customers, depots, and satellites, as well as in their instance category. The instances include:

- 4 different numbers of customers: 15, 30, 50, and 100;
- 3 different combinations of the number of depots and satellites: 2 depots and 3 satellites, 3 depots and 5 satellites, and 6 depots and 4 satellites;
- 4 different instance categories: Ca, Cb, Cc, and Cd.

Each instance is identified by a notation consisting of the category name, an index, the number of depots, the number of satellites, and the number of customers. For example, `Ca1-2,3,15` denotes the first instance of category `Ca`, with 2 depots, 3 satellites, and 15 customers.

Each instance is provided as a text file containing three consecutive blocks of data. For example, the file `Ca1-2,3,15` contains:

1. The first 15 lines contain the customer data. Each line contains, in order, the customer's x coordinate, y coordinate, start of the time window, end of the time window, demand, and service time.
2. The following 3 lines contain the satellite data. Each line contains, in order, the satellite's x coordinate, y coordinate, and service time.
3. The final 2 lines contain the depot data. Each line contains, in order, the depot's x coordinate, y coordinate, and service time.

In this assignment, service times are not relevant and can be ignored. Time-window information is also not relevant for the basic 2E-CVRP. It only becomes relevant if you choose to work on the time-window extension. For more information about the instances, please refer to Dellaert et al. (2019).

Hint: Test your implementation on small instances (that you construct yourself), and verify that the results are correct. It is often good practice to write some verification code to check the correctness of your implementation before running the algorithm on larger instances.

Submission

Summarize your work and findings in a report of at most 10 pages (font size = 11pt, margins = 1 inch). Your report should include:

1. a short introduction that describes the problem, summarizes your solution approach, and highlights the main results;
2. a methodology section that describes the developed algorithm and motivates the main design choices. This should also include a discussion of the AI-assisted design process for the additional destroy and repair operators: summarize the suggestions provided by the AI, explain which suggestions you followed or rejected and why, and describe any adaptations you made before implementing the operators;
3. a results section that presents and discusses the results of a computational study. Your experiments should benchmark your developed ALNS against the basic ALNS implementation provided in the template. In particular, discuss how your choices of additional destroy and repair operators, adaptive operator weighting, simulated annealing, and the additional task you selected (alternative constructive heuristic, parameter tuning, or time windows) affect the performance of the algorithm;
4. a short conclusion that summarizes your main findings; and

5. an appendix containing your Python code, the full transcript of your conversation with the AI assistant used to obtain suggestions for the additional destroy and repair operators, a short reflection on how and where you used AI during the assignment beyond this required AI-assisted task, and the name and version of the AI model(s) used. The appendix does not count towards the page limit.

Submit both:

1. the report in *PDF format*; and
2. a *zip-file* containing your Python code. The code must be submitted as plain Python (.py) files; Jupyter notebooks (.ipynb) are not accepted.

Assessment

This assignment will be assessed based on your solution approach (0–5 points) and your report (0–5 points). For the solution approach, we will consider:

- the correctness and functionality of the implemented improvements (4 points);
- the effectiveness of the resulting algorithm in terms of the quality of the obtained solutions ($\frac{1}{2}$ point); and
- the efficiency in terms of the computational resources required to obtain these solutions ($\frac{1}{2}$ point).

For the report, we will consider:

- the use of scientifically appropriate language (1 point);
- the clarity and structure of the exposition (1 point); and
- the quality and depth of the discussion and analysis of the algorithm's performance (3 points).

As part of the assessment, there will be a mandatory short oral discussion of approximately 15 minutes per group. During this discussion, you will be asked to explain your implementation, design choices, and observations from the computational experiments. The discussion is intended to assess your understanding of your submitted work and may inform the assessment.

The oral discussions will take place on **Wednesday, October 14, 13:30–17:15**. Each group must sign up for one of the 15-minute slots posted in Canvas. You must participate in the oral discussion to complete the assignment.

The total number of points obtained (out of 10) directly translates to your grade for this assignment.

Questions

If you have questions about the assignment, please use the designated discussion board on Canvas.

References

Dellaert, N., F. D. Saridarq, T. Van Woensel, and T. G. Crainic (2019). "Branch-and-price–based algorithms for the two-echelon vehicle routing problem with time windows." In: *Transportation Science* 53.2, pp. 463–479. DOI: 10.1287/trsc.2018.0844.

Ropke, S. and D. Pisinger (2006). "An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows." In: *Transportation Science* 40.4, pp. 455–472. DOI: 10.1287/trsc.1050.0135.

Sluijk, N., A. M. Florio, J. Kinable, N. Dellaert, and T. Van Woensel (2023). "Two-echelon vehicle routing problems: A literature review." In: *European Journal of Operational Research* 304.3, pp. 865–886. DOI: 10.1016/j.ejor.2022.02.022.